



ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Discrete Mathematics

PART 1

COMBINATORIAL THEORY

(Lý thuyết tổ hợp)

PART 2

GRAPH THEORY

(Lý thuyết đồ thị)

Contents of Part 1

Chapter 0: Sets, Relations

Chapter 1: Counting problem

Chapter 2: Existence problem

Chapter 3: Enumeration problem

Chapter 4: Combinatorial optimization problem

CONTENTS

- 1. Introduction to problem**
2. Brute force
3. Branch and bound

1. Introduction to problem

1.1. General problem

1.2. Traveling salesman problem

1.3. Knapsack problem

1.4. Bin backing problem

1.1. General problem

- In many practical application problems of combinatorics, each configuration is assigned to a value equal to the rating of the worth using of the configuration for a particular use purpose.
- Then it appears the problem: Among possible combination configurations, determine the one that the worth using is the best. Such kind of problems is called **the combinatorial optimization problem**.

State the combinatorial optimization problem

The combinatorial optimization problem in general could be stated as follows:

Find the min (or max) of function

$$f(x) \rightarrow \min (\max),$$

with condition:

$$x \in D,$$

where D is a finite set.

Terminologies:

- $f(x)$ – objective function of problem,
- $x \in D$ - a solution
- D – set of solutions of problem.
- Set D is often described as a set of combinatorial configurations that satisfy given properties.
- Solution $x^* \in D$ having minimum (maximum) value of the objective function is called **optimal solution**, and the value $f^* = f(x^*)$ is called **optimal value** of the problem.

1. Introduction to problem

1.1. General problem

1.2. Traveling salesman problem

1.3. Knapsack problem

1.4. Bin backing problem

Traveling Salesman Problem – TSP

(Bài toán người du lịch)

- A salesman wants to travel n cities: $1, 2, 3, \dots, n$.
- *Itinerary is a way of starting from a city, and going through all the remaining cities, each city exactly once, and then back to the starting city.*
- Given c_{ij} is the cost of going from city i to city j ($i, j = 1, 2, \dots, n$),
- Find the itinerary with minimum total cost.

Traveling Salesman Problem – TSP

We have a 1-1 correspondence between a *itinerary*

$$T_{\pi(1)} \rightarrow T_{\pi(2)} \rightarrow \dots \rightarrow T_{\pi(n)} \rightarrow T_{\pi(1)}$$

and a permutation $\pi = (\pi(1), \pi(2), \dots, \pi(n))$ of n natural numbers 1, 2, ..., n .

Set the cost of itinerary:

$$f(\pi) = c_{\pi(1),\pi(2)} + \dots + c_{\pi(n-1),\pi(n)} + c_{\pi(n),\pi(1)}.$$

Denote:

Π - set of all permutations of n natural numbers 1, 2, ..., n .

Traveling Salesman Problem – TSP

- Then, the TSP could be stated as the following combinatorial optimization problem:

$$\min \{ f(\pi) : \pi \in \Pi \}.$$

- One could see that the number of possible itineraries is $n!$, but there are only $(n-1)!$ itineraries if the starting city is fixed.

1. Introduction to problem

1.1. General problem

1.2. Traveling salesman problem

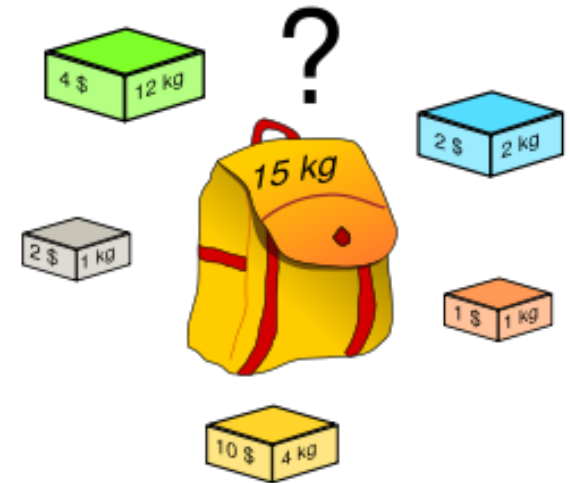
1.3. Knapsack problem

1.4. Bin backing problem

1.3. Knapsack Problem

- Problem Definition

- Want to carry essential items in one bag
- Given a set of items, each has
 - An weight (i.e., 12kg)
 - A value (i.e., 4\$)



- Goal

- To determine the # of each item to include in a collection so that
 - The total weight is less than **some given weight that the bag can carry**
 - And the total value is **as large as possible**

1.3. Knapsack Problem

- Three Types:
 - **0/1 Knapsack Problem**
 - restricts the number of each kind of item to zero or one
 - **Bounded Knapsack Problem**
 - restricts the number of each item to a specific value
 - **Unbounded Knapsack Problem**
 - places no bounds on the number of each item
- Complexity Analysis
 - The general knapsack problem is known to be **NP-hard**
 - No polynomial-time algorithm is known for this problem

0/1 Knapsack Problem

- Problem: John wishes to take n items on a trip
 - The weight of item i is w_i and items are all different (0/1 Knapsack Problem)
 - The items are to be carried in a knapsack whose weight capacity is c
 - When sum of item weights $\leq c$, all n items can be carried in the knapsack
 - When sum of item weights $> c$, some items must be left behind
- Which items should be taken/left?



0/1 Knapsack Problem

- John assigns a profit p_i to item i
 - All weights and profits are positive numbers
- John wants to select a subset of the n items to take
 - The weight of the subset should not exceed the capacity of the knapsack (constraint)
 - Cannot select a fraction of an item (constraint)
 - The profit of the subset is the sum of the profits of the selected items (optimization function)
 - The profit of the selected subset should be maximum (optimization criterion)
- Let $x_i = 1$ when item i is selected and $x_i = 0$ when item i is not selected
 - Because this is a 0/1 Knapsack Problem, you can choose the item or not choose it.

0/1 Knapsack Problem

- A subset of items that John can carry with him can be represented by a binary vector of length n : $x = (x_1, x_2, \dots, x_n)$, where $x_j = 1$ when item j is selected and $x_j = 0$ when item j is not selected, $j = 1, \dots, n$
- For each solution x , the profit of carried items is

$$f(x) = \sum_{j=1}^n c_j x_j,$$

The weight of carried items is

$$g(x) = \sum_{j=1}^n a_j x_j$$

0/1 Knapsack Problem

0/1 Knapsack problem could be stated in the form of the following combinatorial optimization problem:

Among binary vectors of length n that satisfy the condition $g(x) \leq b$, determine the vector x^* giving the maximum value of objective function $f(x)$:

$$\max \{ f(x): x \in A^n, g(x) \leq b \}.$$

$$A^n = \{(a_1, \dots, a_n): a_i \in \{0, 1\}, i=1, 2, \dots, n\}.$$

1. Introduction to problem

1.1. General problem

1.2. Traveling salesman problem

1.3. Knapsack problem

1.4. Bin backing problem

1.4. Bin packing

- Given n items: the weight of items are $w_1, w_2, \dots, w_n$. Need to find a way to place these n items into bins of same size b such that the number of bins used is minimal.

- We have the assumption:

$$w_i \leq b, i = 1, 2, \dots, n.$$

- Therefore, the number of bins needed to hold all these n items is not more than n . The problem is to find the minimum possible number of bins:
 - We will give user n bins. The problem is to help user to determine: each of the n item will be placed in which of the n bins, so that the number of bins having items in them is minimum.

1.4. Bin packing

- We have the boolean variable

$x_{ij} = 1$, if item i is placed in bin j ,
0, otherwise.

Then, bin packing problem is stated in the form:

$$\sum_{j=1}^n \text{sgn}\left(\sum_{i=1}^n x_{ij}\right) \rightarrow \min,$$

$$\sum_{j=1}^n x_{ij} = 1, \quad i = 1, 2, \dots, n$$

$$\sum_{i=1}^n w_i x_{ij} \leq b, \quad j = 1, 2, \dots, n;$$

Note: The signum function of a real number x is defined as follows:

$$\text{sgn}(x) := \begin{cases} -1 & \text{if } x < 0, \\ 0 & \text{if } x = 0, \\ 1 & \text{if } x > 0. \end{cases}$$

CONTENTS

1. Introduction to problem
- 2. Brute force**
3. Branch and bound

Method description

- One of the most obvious methods to solve the combinatorial optimization problem is: On the basis of the combinatorial enumeration algorithms, we go through each solution of the problem, and for each solution, we calculate its value of objective function; then compare values of objective functions of all solutions to find the optimal solution whose objective function is minimal (maximal).
- The approach based on such principles is called the brute force.

Example: 0/1 knapsack problem

- Consider 0/1 knapsack problem

$$\max \{f(x) = \sum_{j=1}^n v_j x_j : x \in D\},$$

$$\text{where } D = \{x = (x_1, x_2, \dots, x_n) \in A^n : \sum_{j=1}^n w_j x_j \leq b\}$$

- v_j, w_j, b are positive integers, $j=1, 2, \dots, n$.
- Need algorithm to enumerate all elements of set D

Backtracking: enumerate all possible solutions

- **Construct set S_k :**

- $S_1 = \{0, t_1\}$, where $t_1 = 1$ if $b \geq w_1$; $t_1 = 0$, otherwise
- Assume the current partial solution is $(x_1, \dots, x_{k-1})$. Then:

- The remaining capacity of the bag is:

$$b_{k-1} = b - w_1x_1 - \dots - w_{k-1}x_{k-1}$$

- The value of items already in the bag is:

$$f_{k-1} = v_1x_1 + \dots + v_{k-1}x_{k-1}$$

Therefore: $S_k = \{0, t_k\}$, where $t_k = 1$ if $b_{k-1} \geq w_k$; $t_k = 0$, otherwise

- **Implement S_k ?**

for ($y = 0$; $y++$; $y \leq t_k$)

Program in pseudo code

```
int x[20], xopt[20], c[20], w[20];  
int n,b, bk, fk, fopt;
```

```
void InputData ( )
```

```
{  
    <Enter value of n, v, w, b>;  
}
```

```
void PrintSolution ( )
```

```
{  
    <Optimal solution: xopt;  
    Optimal value of objective  
    function: fopt>;  
}
```

```
int main( )
```

```
{  
    InputData( );  
    bk=b;  
    fk= 0;  
    fopt= 0;  
    BackTrack(1);  
    PrintSolution( );  
}
```

```
void BackTrack(int k)
{
    int j, t;
    if (bk>=w[k]) t=1 else t=0;
    for (j= t; j--; j>=0)
    {
        x[k] = j;
        bk= bk-w[k]*x[k];
        fk= fk + v[k]*x[k];
        if (k == n)
        {
            if (fk>fopt) {
                xopt:=x; fopt:=fk;
            }
        }
        else BackTrack(k+1);
        bk = bk+w[k]*x[k];
        fk = fk - v[k]*x[k];
    }
}
```

Comment

- Brute force is difficult to do even on the most modern super computer. Example to enumerate all

$$15! = 1\,307\,674\,368\,000$$

permutations on the machine with the calculation speed of 1 billions operations per second, and if to enumerate one permutation requires 100 operations, then we need 130767 seconds > 36 hours!

$$20! \implies 7645 \text{ years}$$

Comment

- However, it must be emphasized that in many cases (for example, in the traveling salesman problem, the knapsack, bin packing problem), we have not found yet any effective methods other than the brute force.

Comment

- Then, a problem arises that in the process of enumerating all solutions, we need to make use of the found information to eliminate solutions that are definitely not optimal.
- In the next section, we will look at such a search approach to solve the combinatorial optimization problems. In literature, it is called **Branch and bound algorithm**.

CONTENTS

1. Introduction to problem
2. Brute force
- 3. Branch and bound**

3. Branch and bound

3.1. General diagram

3.2. Example

3.2.1. Traveling salesman problem

3.2.2. Knapsack problem

3.1. General diagram

- Branch and bound algorithm consists of 2 procedures:
 - Branching Procedure
 - Bounding Procedure
- **Branching procedure:** The process of partitioning the set of solutions into subsets of size decreasing gradually until the subsets consists only one element.
- **Bounding procedure:** It is necessary to give an approach to calculate the bound for the value of the objective function on each subset A in the partition of the set of solutions.

3.1. General diagram

- We will describe the idea of algorithm on the model of the following general combinatorial optimization problem:

$$\min \{ f(x) : x \in D \},$$

where D is the finite set.

- Assume set D is described as following:

$$D = \{ x = (x_1, x_2, \dots, x_n) \in A_1 \times A_2 \times \dots \times A_n : \\ x \text{ satisfies property } P \},$$

where $A_1, A_2, \dots, A_n$ are finite set, and P is property on the Descartes product $A_1 \times A_2 \times \dots \times A_n$.

Comment

- The requirement about describe the set D is to be able to use the backtracking algorithm to enumerate all solutions of the problem.
- Problem

$$\max \{f(x): x \in D\}$$

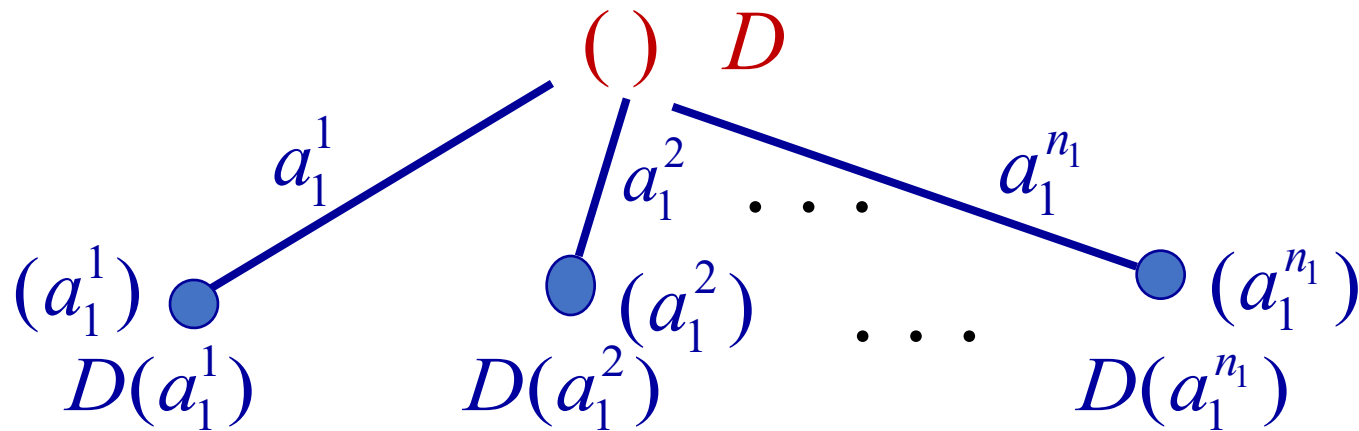
is the equivalent of the problem

$$\min \{g(x): x \in D\}, \text{ where } g(x) = -f(x)$$

Therefore, we can limit to considering the minimize problem

Branching procedure

Branching procedure can implement by using backtracking:



where $D(a_1^i) = \{x \in D : x_1 = a_1^i\}$, $i = 1, 2, \dots, n_1$

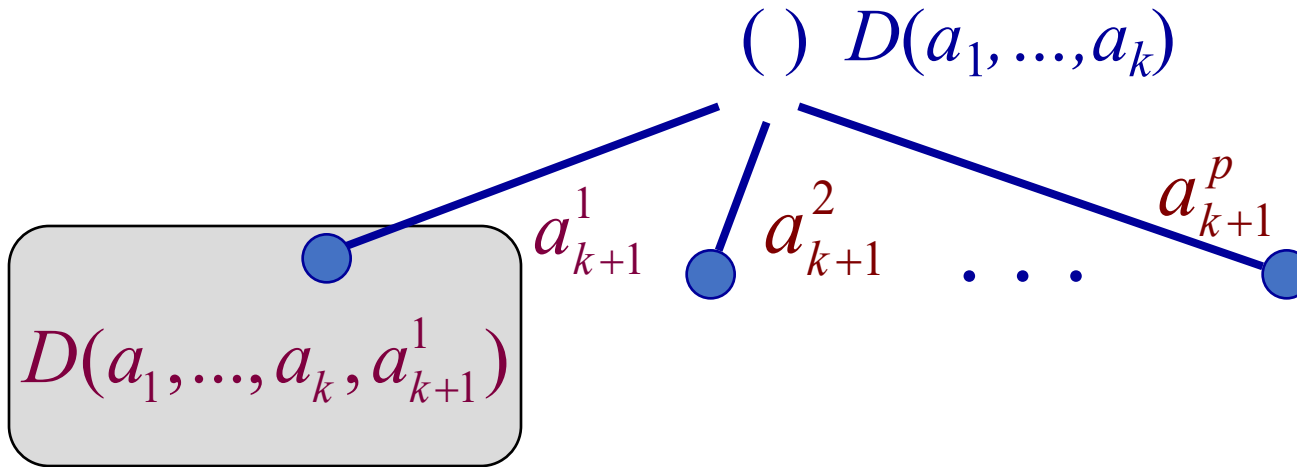
is the set of solutions which can be obtained from partial solution (a_1^i)

We have the partition:

$$D = D(a_1^1) \cup D(a_1^2) \cup \dots \cup D(a_1^{n_1})$$

Branching

- Branching can be described as following:



➤ We have partition:
$$D(a_1, \dots, a_k) = \bigcup_{i=1}^p D(a_1, \dots, a_k, a_{k+1}^i)$$

$D(a_1, \dots, a_k) = \{ x \in D : x_i = a_i, i = 1, \dots, k \}$ is a subset of solutions where the first k elements of solutions are already known: $x_1 = a_1, x_2 = a_2, \dots, x_k = a_k$

Bounding

- We need to determine function g defined on the set of all partial solutions that satisfies the following inequality:

$$g(a_1, \dots, a_k) \leq \min \{f(x) : x \in D(a_1, \dots, a_k)\} \quad (*)$$

The value of objective function of all solutions having the first k elements as $(a_1, a_2, \dots, a_k)$

For each k -level partial solution $(a_1, a_2, \dots, a_k)$, $k = 1, 2, \dots$

- The inequality $(*)$ means that the value of g of partial solution $(a_1, a_2, \dots, a_k)$ is not greater than the minimum value of objective function of solution set $D(a_1, \dots, a_k)$

$$D(a_1, \dots, a_k) = \{x \in D : x_i = a_i, i = 1, \dots, k\},$$

In other word, $g(a_1, a_2, \dots, a_k)$ is the **lower bound** of the value of objective function of solution set $D(a_1, a_2, \dots, a_k)$.

Cut branch by using lower bound

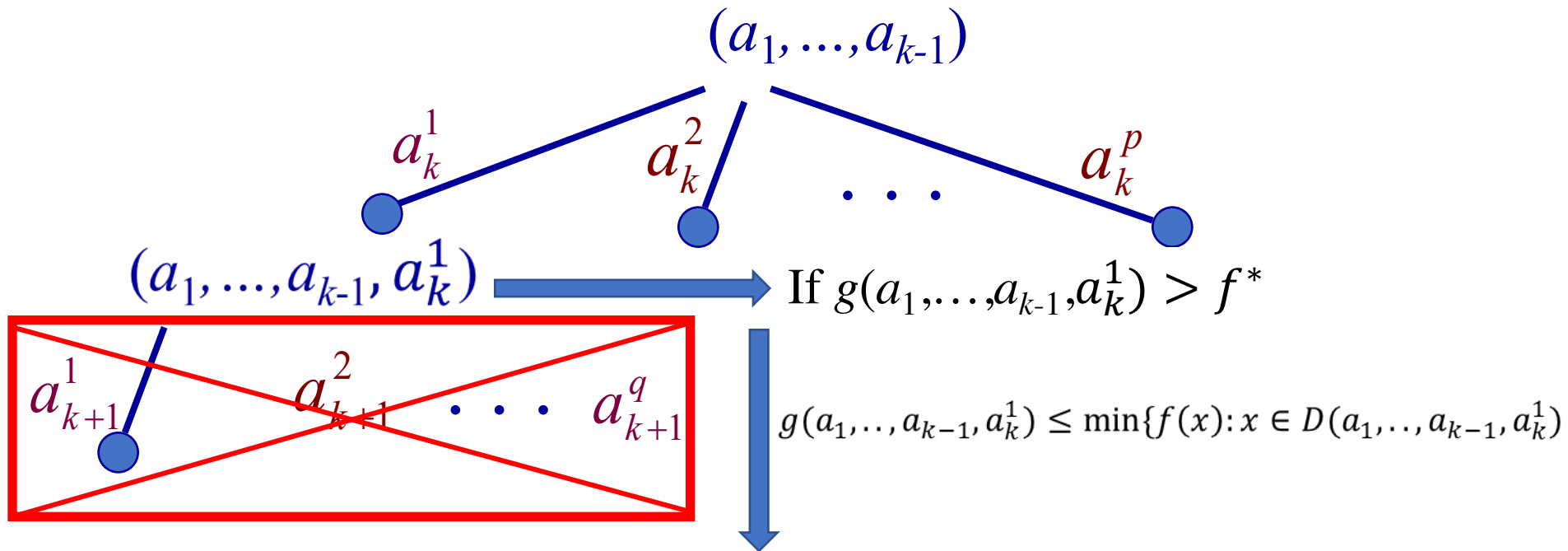
- Assume we already have function g defined as above. We will use this function to reduce the amount of searching during the process to consider all possible solutions in the backtracking algorithm.
- In the process to enumerate solutions, assume we already obtain some solutions. Thus, denote x^* the solution with objective function is minimum among all solutions obtained so far, and denote $f^* = f(x^*)$
- We call
 - x^* is the current best solution (optimal solution),
 - f^* is the current best value of objective function (optimal objective value).

Cut branch by using lower bound

f^* is the current best objective value

$$\text{If } g(a_1, \dots, a_{k-1}, a_k^2) > f^*$$

$$\text{If } g(a_1, \dots, a_{k-1}, a_k^p) > f^*$$



$\rightarrow$ all solutions with first k elements as $(a_1, \dots, a_{k-1}, a_k^1)$ certainly have the objective value $> f^*$ $\rightarrow$ we do not need to browse this branch

Branch and bound

```
void Branch(int k) {
    //Construct  $x_k$  from partial solution  $(x_1, x_2, \dots, x_{k-1})$ 

    for  $a_k \in A_k$ 
        if  $(a_k \in S_k)$ 
        {
             $x_k = a_k$ ;

            if  $(k == n)$  <Update Record>;

            else if  $(g(x_1, \dots, x_k) \leq f^*)$  Branch(k+1);
        }
    }

void BranchAndBound ( ) {
     $f^* = +\infty$ ;

    //if you know any solution  $x^*$  then set  $f^* = f(x^*)$ 

    Branch(1);

    if  $(f^* < +\infty)$ 
        < $f^*$  is the optimal objective value,  $x^*$  is optimal solution >

    else < problem does not have any solutions >;
}
```

Note:

$$g(a_1, \dots, a_k) \leq \min \{f(x): x \in D(a_1, \dots, a_k)\} \quad (*)$$

The construction of g function depends on each specific combinatorial optimization problem. Usually we try to build it so that:

- Calculating the value of g must be simpler than solving the combinatorial optimization problem on the right side of (*).
- The value of $g(a_1, \dots, a_k)$ must be close to the value of the right side of (*).

Unfortunately, these two requirements are often contradictory in practice.

3. Branch and bound

3.1. General diagram

3.2. Example

3.2.1. Traveling salesman problem

3.2.2. Knapsack problem



Sir William Rowan Hamilton
1805 - 1865

Traveling Salesman Problem – TSP

- A salesman wants to travel n cities: $1, 2, 3, \dots, n$.
- *Itinerary is a way of starting from a city, and going through all the remaining cities, each city exactly once, and then back to the starting city.*
- Given c_{ij} is the cost of going from city i to city j ($i, j = 1, 2, \dots, n$),
- Find the itinerary with minimum total cost.

3.2.1. Traveling salesman problem

Fix the starting city as city 1, the TSP leads to the problem:

- Determine the minimum value of

$$f(1, x_2, \dots, x_n) = c[1, x_2] + c[x_2, x_3] + \dots + c[x_{n-1}, x_n] + c[x_n, 1]$$

where

$(x_2, x_3, \dots, x_n)$ is permutation of natural numbers $2, \dots, n$.

Lower bound function

- Denote

$$c_{min} = \min \{ c[i, j] , i, j = 1, 2, \dots, n, i \neq j \}$$

the smallest cost between all pairs of cities.

- We need to evaluate the lower bound for the partial solution $(1, u_2, \dots, u_k)$ corresponding to the partial journey that has passed through k cities

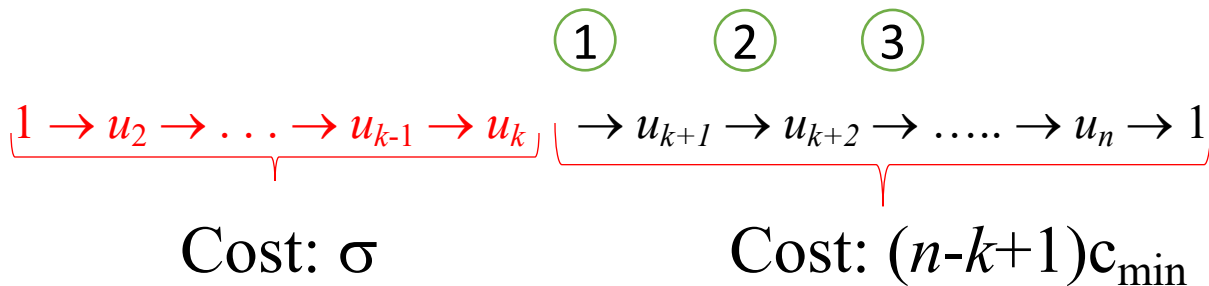
$$1 \rightarrow u_2 \rightarrow \dots \rightarrow u_{k-1} \rightarrow u_k$$

Lower bound function

- The cost need to pay for this partial solution is

$$\sigma = c[1, u_2] + c[u_2, u_3] + \dots + c[u_{k-1}, u_k].$$

- To develop it as the complete journey:



We still need to go through $n-k+1$ segments, each segment with the cost at least $c_{\min}$, thus the lower bound of the partial solution $(1, u_2, \dots, u_k)$ can be calculated by the formula:

$$g(1, u_2, \dots, u_k) = \sigma + (n-k+1) c_{\min}$$

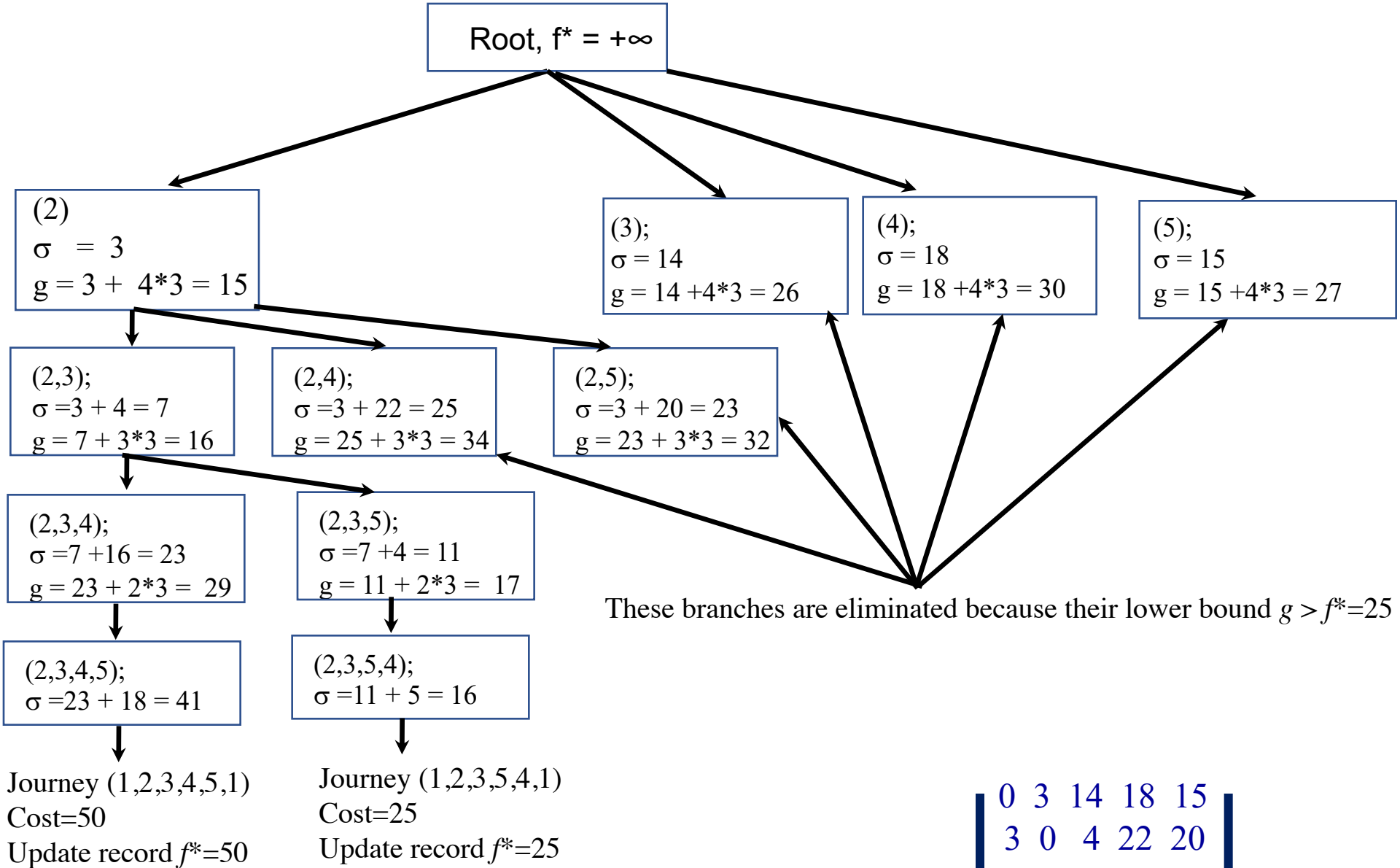
Example

Give 5 cities $\{1, 2, 3, 4, 5\}$. Solve the TSP where the salesman starts from the city 1, and the cost matrix:

$$C = \begin{bmatrix} 0 & 3 & 14 & 18 & 15 \\ 3 & 0 & 4 & 22 & 20 \\ 17 & 9 & 0 & 16 & 4 \\ 9 & 20 & 7 & 0 & 18 \\ 9 & 15 & 11 & 5 & 0 \end{bmatrix}$$

Example

- We have $c_{min} = 3$. The process executing the algorithm is described by the solution search tree.
- Information written in each box is the following in order:
 - elements of partial solution,
 - σ is the cost of partial solution,
 - g – lower bound of partial solution.



$$C = \begin{bmatrix} 0 & 3 & 14 & 18 & 15 \\ 3 & 0 & 4 & 22 & 20 \\ 17 & 9 & 0 & 16 & 4 \\ 9 & 20 & 7 & 0 & 18 \\ 9 & 15 & 11 & 5 & 0 \end{bmatrix}$$

Result

Terminate the algorithm, we obtain:

- Optimal solution $(1, 2, 3, 5, 4, 1)$ correspond to the journey

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 1 ,$$

- The minimum cost is 25.

Implement

```
void Branch(int k)
{
    for (int v = 2; v<=n;v++) {
        if (visited[v] == FALSE) {
             $x_k = v$ ; visited[v] = TRUE;
             $f = f + c(x_{k-1}, x_k)$ ;
            if (k == n) //Update record:
                { if ( $f + c(x_n, x_1) < f^*$ )  $f^* = f + c(x_n, x_1)$ ; }
            else {
                 $g = f + (n-k + 1)*cmin$ ; //calculate bound
                if ( $g < f^*$ ) Branch(k + 1);
            }
        }
         $f = f - c(x_{k-1}, x_k)$ ;
        visited[v] = FALSE;
    }
}
```

```
void Branch(int k)
{
    //Construct  $x_k$  from partial solution ( $x_1, x_2, \dots, x_{k-1}$ )
    for  $a_k \in A_k$ 
        if ( $a_k \in S_k$ )
        {
             $x_k = a_k$ ;
            if (k == n) <Update Record>;
            else if ( $g(x_1, \dots, x_k) \leq f^*$ ) Branch(k+1);
        }
}
```

Implement

```
void BranchAndBound()
```

```
{
```

```
    f* = +∞;
```

```
    for (v = 1; v<=n;v++) visited[v] = FALSE;
```

```
    f=0; x1 = 1; visited[x1] = TRUE;
```

```
    Branch(2);
```

```
    return f*;
```

```
}
```

```
void BranchAndBound ( )
```

```
{
```

```
    f* = +∞;
```

```
    //if you know any solution x* then set f* = f(x*)
```

```
    Branch(1);
```

```
    if (f* < +∞)
```

```
        <f* is the optimal objective value, x* is optimal solution >
```

```
    else < problem does not have any solutions >;
```

```
}
```

3. Branch and bound

3.1. General diagram

3.2. Example

3.2.1. Traveling salesman problem

3.2.2. Knapsack problem



Sir William Rowan Hamilton
1805 - 1865

3.2.2. Knap sack problem

- There are n types of items.
- Item type j has
 - weight w_j and
 - profit p_j ($j = 1, 2, \dots, n$) .
- We need to select subsets of these items to put it into the bag of capacity c such that the total profit obtained from items loaded in the bag is maximum.

3.2.2. Knap sack problem

- We have the variable

x_j – number items type j loaded in the bag, $j=1,2, \dots, n$

- Mathematical model of problem: Find

$$f^* = \max \{ f(x) = \sum_{j=1}^n p_j x_j : \sum_{j=1}^n w_j x_j \leq c, x_j \in Z_+, j = 1, 2, \dots, n \}$$

where Z_+ is the set of nonnegative integers

Knapsack problem with integer variables

- Denote D the set of solutions to the problem:

$$D = \{ x = (x_1, \dots, x_n) : \sum_{j=1}^n p_j x_j \leq c, x_j \in Z_+, j = 1, 2, \dots, n \}$$

Construct upper bound

- Assume we index the item in the order such that the following inequality is satisfied:

$$v_1 / w_1 \geq v_2 / w_2 \geq \dots \geq v_n / w_n .$$

(it means items are ordered in descending order of profit per one unit of weight)

- To construct the upper bound function, we consider the following Knapsack continuous variables (KPC): Find

$$g^* = \max \left\{ f(x) = \sum_{j=1}^n p_j x_j : \sum_{j=1}^n w_j x_j \leq c, x_j \geq 0, j = 1, 2, \dots, n \right\}$$

Construct upper bound function

Proposition. *The optimal solution to the KPC is vector $(x^* = x_1^*, x_2^*, \dots, x_n^*)$ where elements are determined by the formula:*

$$x_1^* = c/w_1, x_2^* = x_3^* = \dots = x_n^* = 0$$

and the optimal value is $g^ = v_1 b / w_1$.*

Proof. Consider $x = (x_1, \dots, x_n)$ as a solution to the KPC. Then

$$p_j \leq (p_1 / w_1) w_j, j = 1, 2, \dots, n$$

as $x_j \geq 0$, we have

$$p_j x_j \leq (p_1 / w_1) w_j x_j, j = 1, 2, \dots, n.$$

• Therefore

$$\begin{aligned} \sum_{j=1}^n p_j x_j &\leq \sum_{j=1}^n (p_1 / w_1) w_j x_j \\ &= (p_1 / w_1) \sum_{j=1}^n w_j x_j \\ &\leq (p_1 / w_1) c = g^* \end{aligned}$$

Proposition is proved.

Calculate the upper bound

- Now we have the k -level partial solution: $(u_1, u_2, \dots, u_k)$, then the profit of items currently loaded in the bag is

$$\sigma_k = p_1 u_1 + p_2 u_2 + \dots + p_k u_k$$

and the remaining capacity of the bag is

$$c_k = c - (w_1 u_1 + w_2 u_2 + \dots + w_k u_k)$$

- We have:
$$\begin{aligned} & \max \{f(x) : x \in D, x_j = u_j, j = 1, 2, \dots, k\} \\ &= \max \left\{ \sigma_k + \sum_{j=k+1}^n p_j x_j : \sum_{j=k+1}^n w_j x_j \leq c_k, x_j \in \mathbb{Z}_+, j = k+1, k+2, \dots, n \right\} \\ &\leq \sigma_k + \max \left\{ \sum_{j=k+1}^n p_j x_j : \sum_{j=k+1}^n w_j x_j \leq c_k, x_j \geq 0, j = k+1, k+2, \dots, n \right\} \\ &= \sigma_k + p_{k+1} c_k / w_{k+1} \end{aligned}$$

- Thus, we can calculate the upper bound for the partial solution $(u_1, u_2, \dots, u_k)$ by formula $g(u_1, u_2, \dots, u_k) = \sigma_k + p_{k+1} c_k / w_{k+1}$

Calculate the upper bound

- **Note:** When continuing build the $(k+1)$ th element of solution, candidates for x_{k+1} are $0, 1, \dots, [c_k / w_{k+1}]$
- Using the result of the proposition, when selecting value for x_{k+1} , we browse candidates for x_{k+1} in the descending order: $[c_k / w_{k+1}], [c_k / w_{k+1}] - 1, \dots, 1, 0$

Example

- Solve the knap sack problem using the branch and bound algorithm:

$$\begin{aligned}f(x) &= 10 x_1 + 5 x_2 + 3 x_3 + 6 x_4 \rightarrow \max, \\5 x_1 + 3 x_2 + 2 x_3 + 4 x_4 &\leq 8, \\x_j &\in Z_+, j = 1, 2, 3, 4.\end{aligned}$$

- Note that in this example, all four items are already sorted in descending order of profit on an unit weight

$$\frac{10}{5} = 2 > \frac{5}{3} \approx 1,66 > \frac{3}{2} = 1,5 = \frac{6}{4}$$

Example

- The process executing the algorithm is described by the solution search tree.
- Information written in each box is the following in order:
 - elements of partial solution,
 - σ is the cost of partial solution (profit of items currently loaded in the bag),
 - w : remaining capacity of the bag,
 - g : upper bound of partial solution.

Root, $f^* = -\infty$

$$f(x) = 10x_1 + 5x_2 + 3x_3 + 6x_4 \rightarrow \max,$$
$$5x_1 + 3x_2 + 2x_3 + 4x_4 \leq 8,$$
$$x_j \in \mathbb{Z}^+, j = 1, 2, 3, 4.$$

Select item 1:

$$8/5 = 1$$

$$x_1 = 1$$

$$x_1 = 0$$

(1) $\sigma = 10$

$$w = 8 - 5 = 3; g = 10 + 5 \cdot 3/3 = 15$$

$$x_2 = 1$$

Select item 2:

$$3/3 = 1$$

$$x_2 = 0$$

(1,1); $\sigma = 10 + 5 = 15$

$$w = 3 - 3 = 0; g = 15$$

(1,0); $\sigma = 10 + 0 = 10$

$$w = 3; g = 10 + 3 \cdot 3/2 = 14.5$$

$x_3 = 0$ Select item 3:

$$0/2 = 0$$

(1,1,0); $\sigma = 15$

$$w = 0; g = 15$$

$$x_4 = 0$$

Select item 4:

$$0/4 = 0$$

(1,1,0,0)

$$f^* = 15$$

We obtain a new solution
Update record: $f^* = 15$

Eliminate because upper bound $g < f^* = 15$

Eliminate because upper bound $g < f^* = 15$

• Finish algorithm, we obtain:

- Optimal solution: $x^* = (1, 1, 0, 0)$,
- Optimal objective value: $f^* = 15$.